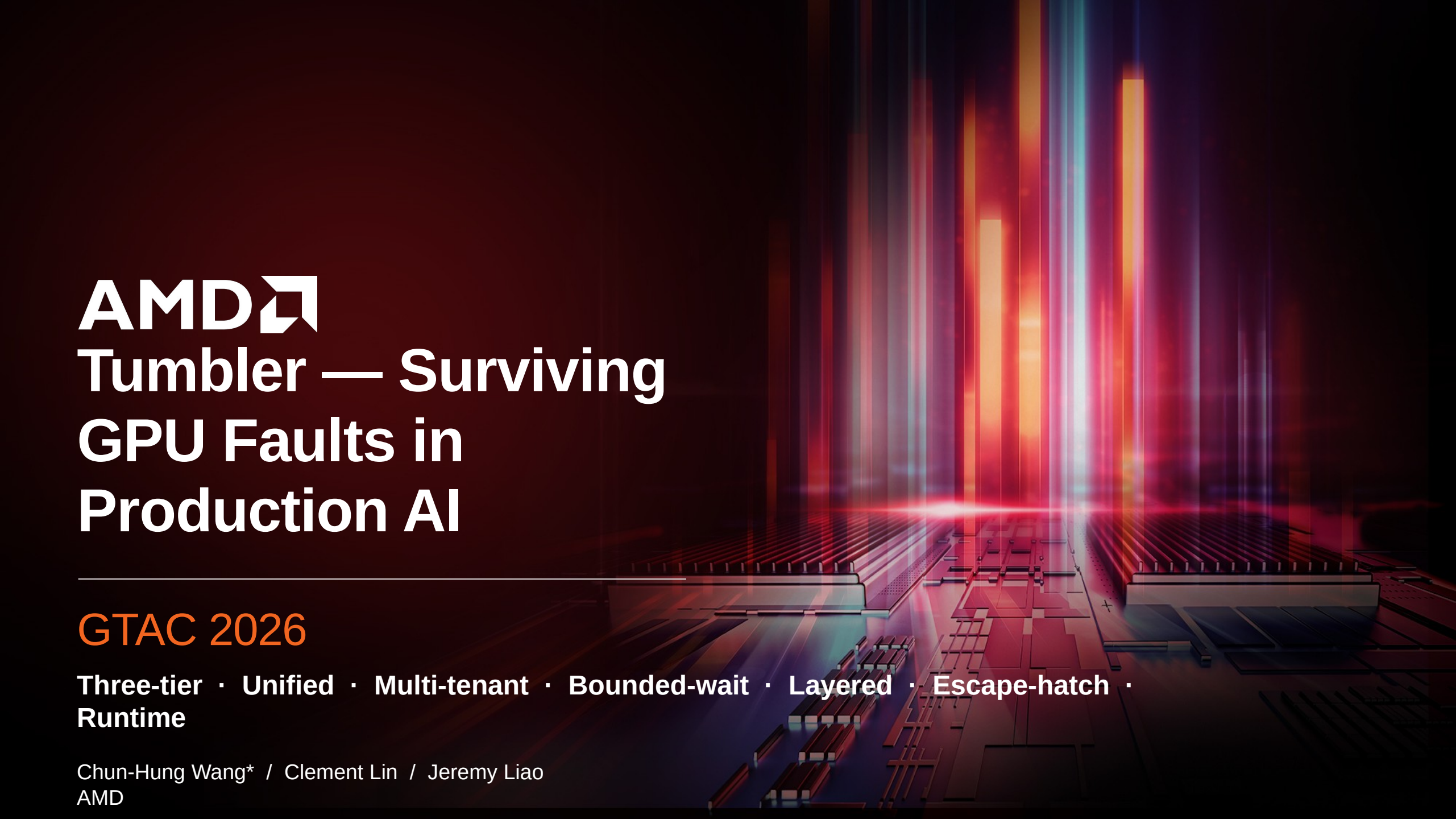


Tumbler — Surviving GPU Faults in Production AI

GTAC 2026

Three-tier · Unified · Multi-tenant · Bounded-wait · Layered · Escape-hatch · Runtime

Chun-Hung Wang* / Clement Lin / Jeremy Liao
AMD

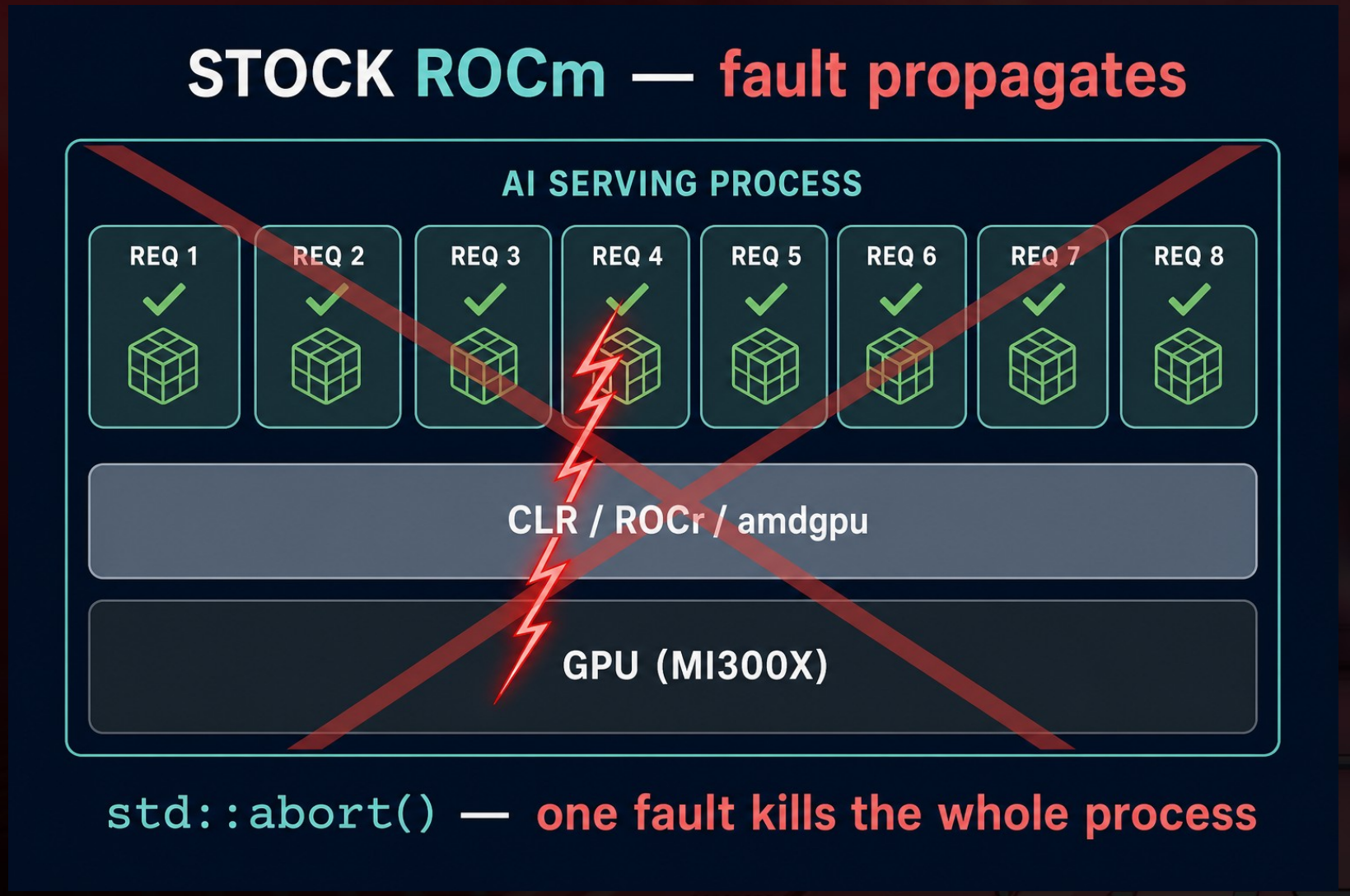
Picture this — but on production AI

If the touchscreen GPU faults on the highway, you want one frozen frame — not the whole infotainment process dead.

Why AI workloads are different from HPC

- HPC: 1 host process owns 1 GPU. A VM fault means data corruption — the only safe action is to abort.
- AI serving: 1 host process owns 100s of concurrent tenant requests sharing the same GPUs, SDMA engines, signals, and KFD pinned memory.
- Same fault, very different cost: one bad request vs one dead serving node taking every co-tenant down with it.
- ROCm was built for the first case. Production AI inverts the assumption — and needs an opt-in escape hatch.

Today — one fault aborts the whole serving process

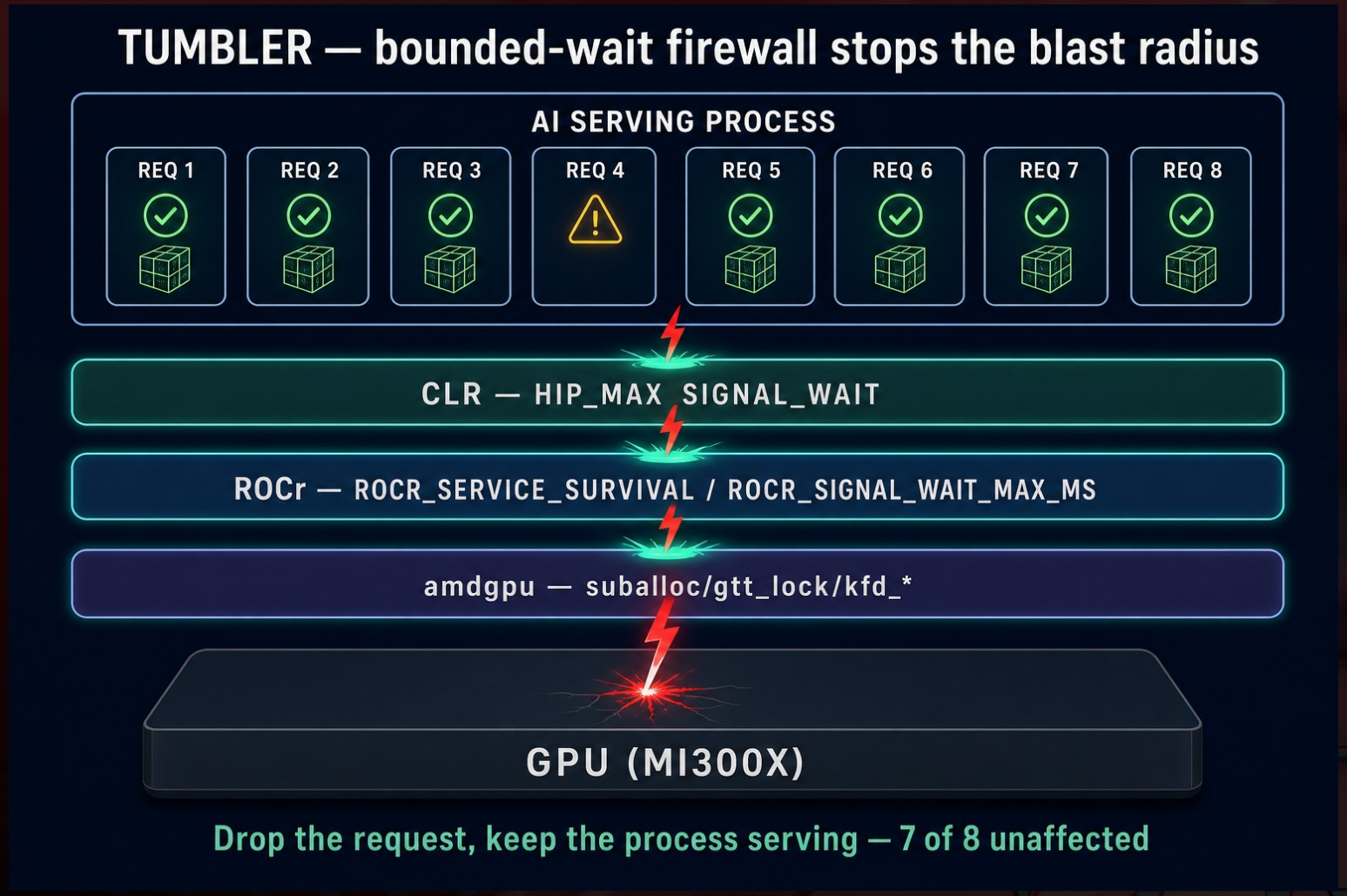


Why ROCm has no built-in survival firewall

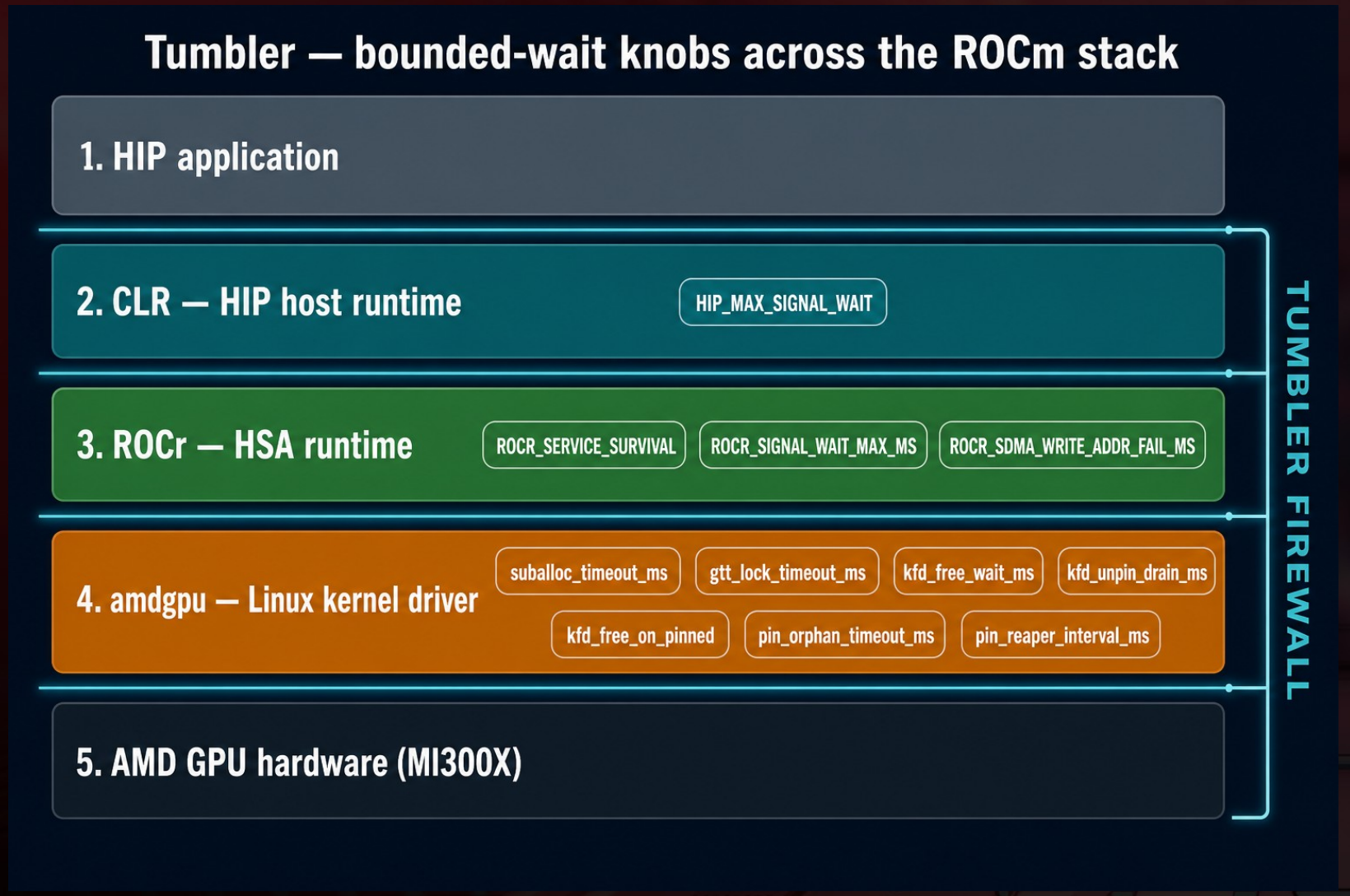
- ROCr inherits the HSA spec: one logical process owns the GPU. Three abort sites in runtime.cpp (memory error / HW exception / VM fault) all call std::abort() unconditionally.
- CLR WaitForSignal() has a 4-sec inner poll, but the outer while loop is unbounded — a wedged signal parks the caller thread forever.
- amdgpu waits with MAX_SCHEDULE_TIMEOUT in kcl_drm_suballoc_new() and on a global mutex in the GTT-window path.
- HSA_SIGNAL_WAIT_ABORT_TIMEOUT exists — but it also aborts the process at the deadline. Right for batch HPC, wrong for AI serving.

TUMBLER — a bounded-wait firewall that stops the blast radius

Three-tier · Unified · Multi-tenant · Bounded-wait · Layered · Escape-hatch · Runtime



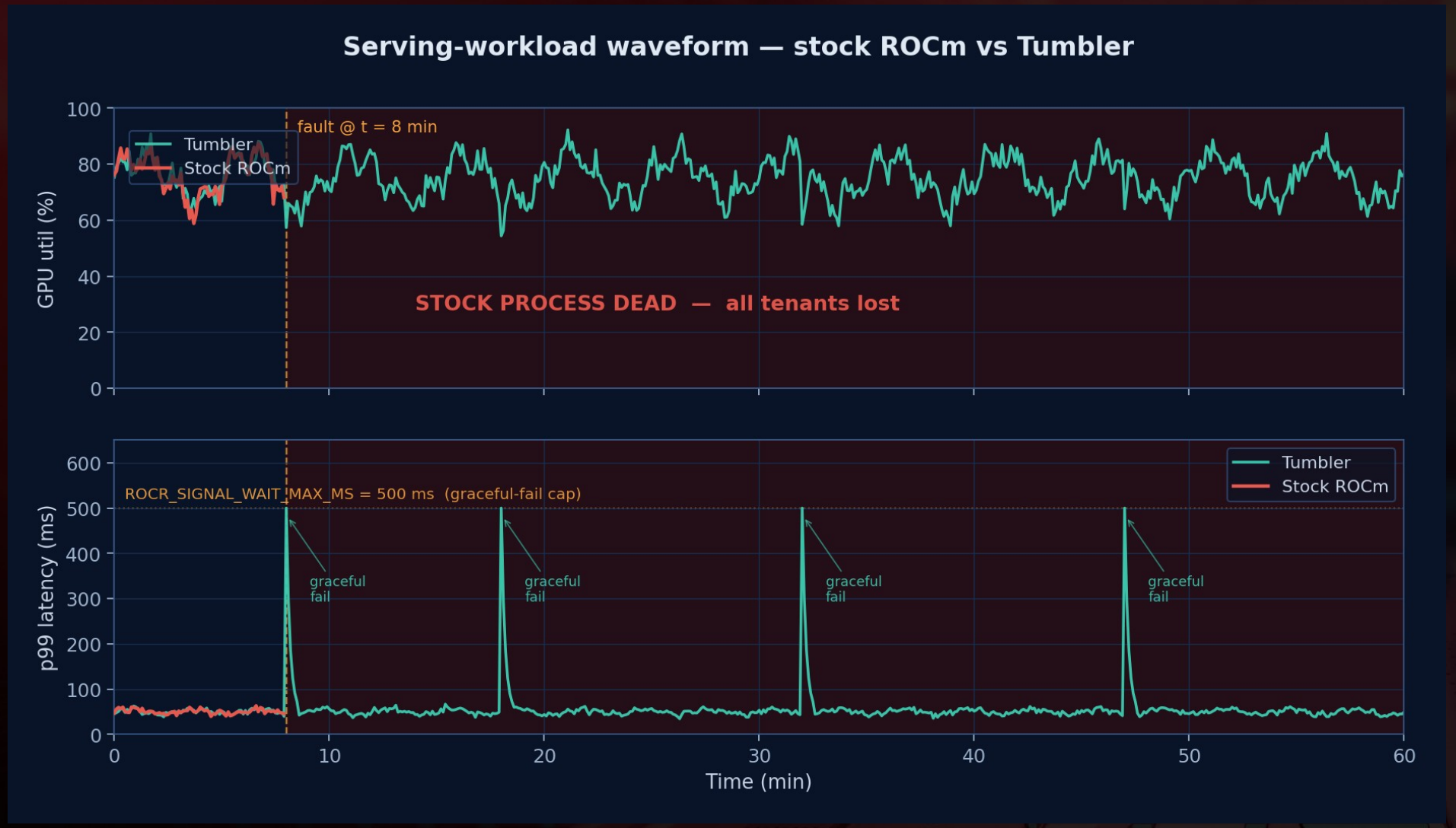
Architecture — knobs across the ROCm stack



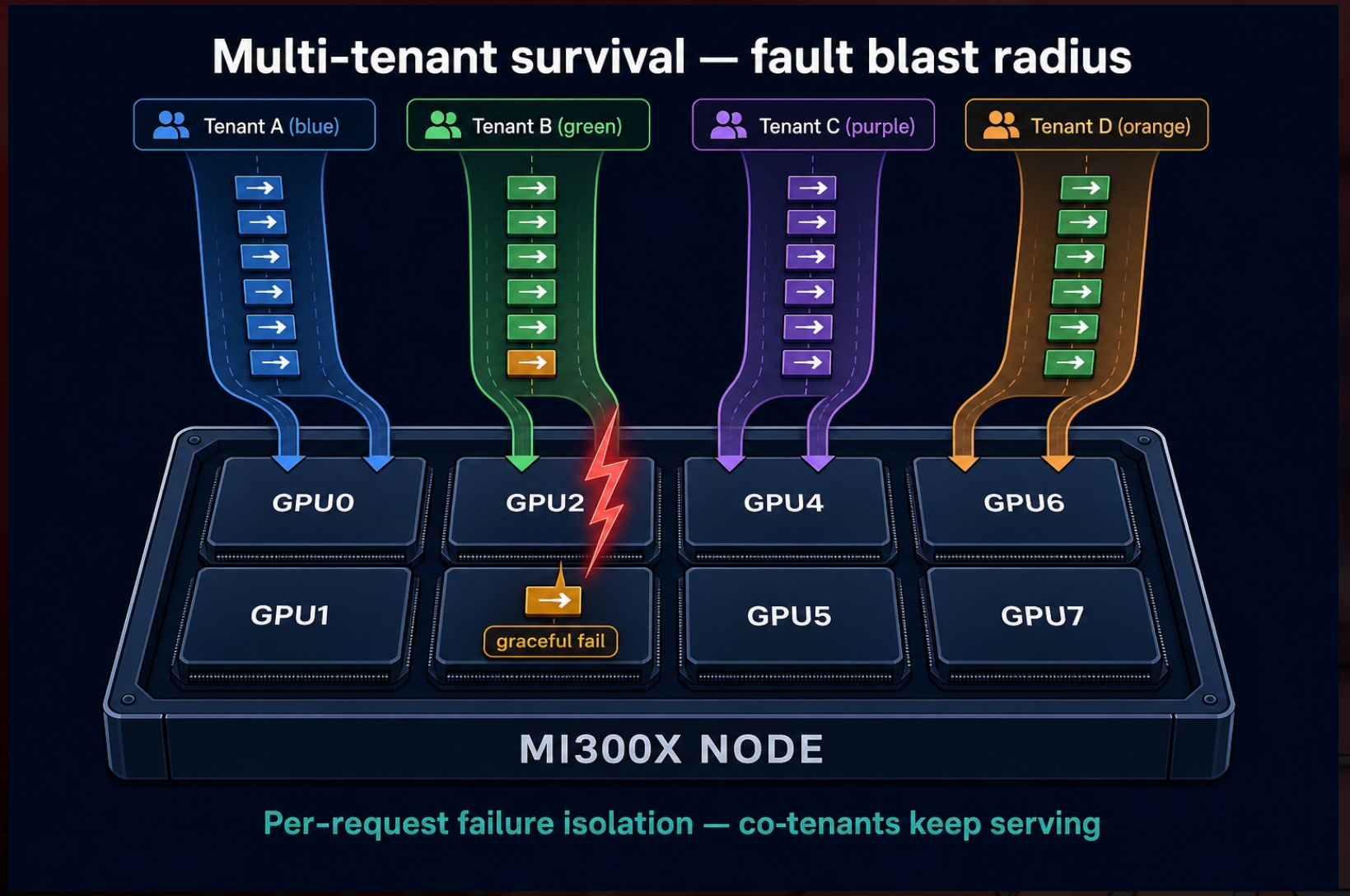
Three layers of defense

Layer	Guards against	How it protects
CLR (HIP host runtime)	A host thread parked forever on a wedged HSA completion signal — the serving scheduler cannot reroute the affected request.	Bounded outer-loop wait on WaitForSignal(). At the deadline the signal is force-completed and the caller returns. The serving framework decides whether to retry or fail the one request.
ROCr (HSA runtime)	std::abort() on the three runtime.cpp abort sites (memory error, non-ECC HW exception, VM fault) — one fault kills the whole process and takes every co-tenant request down with it. Plus SDMA writers spinning forever; InterruptSignal waits parked.	Opt-in non-abort branches at every abort site (ECC stays abort-fatal for data integrity). Bounded caps on the signal-wait inner loop and on the SDMA yield loop. Failure surfaces as an HSA_STATUS error, never as a process abort.
amdgpu (kernel driver)	A wedged SDMA ring holding the global GTT-window mutex and starving every VRAM-touching ioctl on the device. A pinned BO whose owner exited without unpinning permanently holding VRAM. Unbounded dma_fence_wait inside suballoc.	Bounded timeouts at every fence and lock site (suballoc, GTT window, KFD free / unpin). KFD pin-reaper subsystem harvests orphan pins on a deadline. Failure surfaces to user space as -ETIME, never as a hung ioctl.

Production result — single-digit minutes vs healthy waveform



Multi-tenant survival — fault blast radius is one request



Future work — AI-driven survival

- Auto-tune Tumbler deadlines from observed workload signatures (serving-framework tail-latency budget, batch dwell time, in-flight request count).
- Reinforcement-learning-driven survival policy: learn when to fail a request gracefully vs wait out a transient hiccup.
- Feedback channels into the inference framework so the higher-level scheduler can adjust load on graceful-fail events.
- Long-tail items: KFD slow-ioctl bounded path, GTT multi-window lock sharding (deferred for future PRs).

Thank you — questions?

- Chun-Hung Wang — Chun-Hung.Wang@amd.com (corresponding)
- Clement Lin — Clement.Lin@amd.com
- Jeremy Liao — Jeremy.Liao@amd.com

- Code: github.com/AFDEAPAC/Tumbler
- PRs: ROCm/rocm-systems #6328, #6329 / ROCm/amdgpu #214, #215, #216


AMD